

- [Smart DMS v2.0 — Document Management Portal](#)
 - [Table of Contents](#)
 - [Quick Start](#)
 - [Demo Credentials](#)
 - [Plants & Locations](#)
 - [Project Structure](#)
 - [Pages & How They Work](#)
 - [1. Login — / or /login](#)
 - [2. Register — /register](#)
 - [3. Forgot / Reset Password](#)
 - [4. Dashboard — /dashboard](#)
 - [5. Upload Documents — /upload](#)
 - [6. Pending's \(Approvals\) — /approvals](#)
 - [7. Approval Review](#)
 - [8. Document Library — /document-library](#)
 - [9. Master Records \(Plant Assets\) — /plant-assets](#)
 - [10. Customer Records — /customer-records](#)
 - [11. Graphics Report — /graphics-report](#)
 - [12. Revision History — /revision-history](#)
 - [13. Archive — /archive](#)
 - [14. System Log — /system-log](#)
 - [15. People — /people](#)
 - [16. Profile — /profile](#)
 - [17. About DMS — /about](#)
 - [18. About Company — /about/company](#)
 - [Role & Access Control](#)
 - [Database Tables](#)
 - [Services Reference](#)
 - [Environment Variables](#)
 - [Dependencies](#)

Smart DMS v2.0 — Document Management Portal

Company: ZF Rane Automotive India Private Limited **Stack:** Python · Flask · SQLite · Jinja2 · Vanilla JS

Table of Contents

1. [Quick Start](#)
2. [Demo Credentials](#)
3. [Plants & Locations](#)
4. [Project Structure](#)
5. [Pages & How They Work](#)
 - [Login](#)
 - [Register](#)
 - [Forgot / Reset Password](#)
 - [Dashboard](#)
 - [Upload Documents](#)
 - [Pending's \(Approvals\)](#)
 - [Approval Review](#)
 - [Document Library](#)
 - [Master Records \(Plant Assets\)](#)
 - [Customer Records](#)
 - [Graphics Report](#)
 - [Revision History](#)
 - [Archive](#)
 - [System Log](#)
 - [People](#)
 - [Profile](#)
 - [About DMS](#)
 - [About Company](#)
6. [Role & Access Control](#)
7. [Database Tables](#)
8. [Services Reference](#)
9. [Environment Variables](#)
10. [Dependencies](#)

Quick Start

```
# 1. Create and activate virtual environment
python -m venv venv
venv\Scripts\activate           # Windows
source venv/bin/activate        # macOS / Linux
```

```
# 2. Install dependencies
pip install -r requirements.txt

# 3. Run
python app.py
```

Open **http://localhost:5000**

Demo Credentials

Role	Email	Password
Admin	diva@example.com	Pass@12345
Manager (P2)	arun@example.com	Prod@12345
Approver (P3)	sneha@example.com	Eng@12345
User (P4)	rahul@example.com	Safe@12345

You can also log in using the **Employee ID (GENID)** instead of email — e.g. **EMP001** / **Pass@12345**.

Plants & Locations

ID	Plant	Location	Divisions
P1	Trichy Plant	Tiruchirappalli, Tamil Nadu	SGD + OSD
P2	Guduvachery Plant	Guduvachery, Tamil Nadu	SGD + OSD
P3	Guduvachery Plant	Guduvachery, Tamil Nadu	SGD + OSD
P4	Uttarakhand Plant	Rudrapur, Uttarakhand	SGD

Project Structure

```
dms_portal/
├── app.py                                # Flask app factory, blueprint registration
```

- └─ config.py # All config via environment variables
- └─ database.py # SQLite schema creation and migrations
- └─ extensions.py # Flask-Mail instance
- └─ requirements.txt
- └─ smart_dms.db # Main SQLite database (auto-created)
- └─ data/
 - └─ mock_data.py # Seed data: users, plants, customers, records
 - └─ departments.py # Official department list + normalization
 - └─ customers.py # Official customer list + normalization
 - └─ document_library_data.py # Static library category tree
 - └─ smart_dms_users.sqlite3 # User credentials database
- └─ routes/ # One Blueprint per page
 - └─ auth_routes.py
 - └─ dashboard_routes.py
 - └─ upload_routes.py
 - └─ approval_routes.py
 - └─ archive_routes.py
 - └─ document_library_routes.py
 - └─ plant_assets_routes.py
 - └─ customer_records_routes.py
 - └─ graphics_report_routes.py
 - └─ revision_history_routes.py
 - └─ system_log_routes.py
 - └─ people_routes.py
 - └─ profile_routes.py
 - └─ notification_routes.py
 - └─ about_routes.py
 - └─ category_routes.py
- └─ services/ # Business logic, one class per domain
 - └─ auth_service.py
 - └─ document_service.py
 - └─ document_library_service.py
 - └─ category_document_service.py
 - └─ plant_asset_service.py
 - └─ customer_record_service.py
 - └─ revision_history_service.py
 - └─ document_tracking_service.py
 - └─ document_preview_service.py
 - └─ pdf_conversion_service.py
 - └─ system_log_service.py
 - └─ notification_service.py
 - └─ mail_service.py
 - └─ password_reset_service.py
 - └─ user_store_service.py
- └─ templates/
 - └─ layout.html # Shared shell: sidebar + topbar + alerts
 - └─ components/
 - └─ sidebar.html # Navigation sidebar
 - └─ auth/
 - └─ login.html
 - └─ register.html
 - └─ reset_password.html
 - └─ dashboard.html
 - └─ upload.html
 - └─ approvals.html
 - └─ approval_review.html

```
├── document_library.html
├── document_view.html
├── plant_assets.html
├── customer_records.html
├── graphics_report.html
├── revision_history.html
├── archive.html
├── system_log.html
├── people.html
├── profile.html
├── about.html
├── about_company.html

├── static/
│   ├── css/app.css           # Dark industrial theme (IBM Plex fonts)
│   └── js/
│       ├── app.js            # Alert auto-dismiss, sidebar toggle, theme
│       ├── upload.js         # Drag-and-drop, target switching, validation
│       ├── plant_assets.js    # Plant → dept → files AJAX
│       ├── customer_records.js # Customer → files AJAX
│       └── document_library.js # Category browser AJAX
└── uploads/                  # All uploaded files stored here
```

Pages & How They Work

1. Login — / or /login

Route: `auth_routes.py` → `login()`

The entry point of the portal. Accepts either an **email address** or an **Employee ID (GENID)** as the username — the system auto-detects which one was entered based on whether it contains @.

- On successful login, the user's name, email, ID, plant, department, and role are stored in the Flask session.
- A login event is written to the System Log with the user's IP address.
- If already logged in, the user is redirected directly to the Dashboard.
- A "Forgot Password" link on this page triggers a token-based reset email.

2. Register — /register

Route: `auth_routes.py` → `register()`

Self-service account creation. New users fill in:

Field	Notes
Full Name	Free text
Employee ID	Optional GENID
Plant	Dropdown — P1 to P4
Department	Dropdown — official departments only
Email	Must be unique
Password	Minimum 8 characters
Confirm Password	Must match

All new accounts are assigned the **User** role automatically. An admin must manually promote a user to Manager, Approver, or Admin via the database. A register event is written to the System Log.

3. Forgot / Reset Password

Routes: `auth_routes.py` → `forgot_password()` / `reset_password()`

- User submits their email on the login page.
 - If the email exists, a signed time-limited token is generated and a reset link is emailed via Flask-Mail.
 - If SMTP is not configured, the reset link is printed to the terminal (development mode).
 - The reset link opens a form where the user sets a new password (minimum 8 characters).
 - Tokens expire after 1 hour.
-

4. Dashboard — `/dashboard`

Route: `dashboard_routes.py` → `index()`

The main control centre. Shows all uploaded documents in a searchable, filterable table.

Features:

Feature	Detail
Search	Full-text search across file name, document number, revision, uploader, customer, department, plant, status — ranked by relevance score
Filters	Plant, Department, Customer dropdowns
Status badges	Approved (green) · Pending (yellow) · Rejected (red)
Document view	Click any row to open the document viewer
Delete	Admin/Manager/Approver/Supervisor only — soft-deletes to Archive
Bookmarks	Star icon on each row — saved per user
Recently Viewed	Sidebar panel showing last 5 documents opened
Upload trend chart	Bar chart of uploads over the last 90 days
Stats cards	Total · This Month · This Week counts
CSV Export	Downloads the current filtered view as a CSV file

All data is loaded from the `documents` table in `smart_dms.db`. If the database is empty, mock seed data is shown as a placeholder.

5. Upload Documents — `/upload`

Route: `upload_routes.py` → `index()`

A step-by-step form for submitting documents into the system.

Step 1 — Destination

Two radio card options:

- **Approval Queue** — document goes to Pending's for review. Category field is hidden.
- **Document Library** — document is saved directly into a library category. Category and Subcategory fields appear.

Step 2 — File Selection

Drag-and-drop zone or click-to-browse. Accepts .pdf, .docx, .doc, .xlsx, .xls, .pptx, .ppt. Maximum 100 MB per file. Multiple files can be selected at once. File type is validated client-side with colour-coded icons (red=PDF, blue=Word, green=Excel, orange=PPT).

Step 3 — Document Details

Field	Required	Notes
Document Number	Yes	e.g. DOC-2026-014
Revision Number	No	Only fill if this is a revised document
Plant	Auto-filled	From user profile, can be changed
Department	Auto-filled	From user profile, can be changed
Customer	Yes (unless Internal)	Select from official customer list
Internal Document toggle	—	Disables customer field, sets customer to "Internal"
Category	Yes (Library only)	Appears only when Document Library is selected
Subcategory	No	Appears only when Document Library is selected

Step 4 — Revision Summary (optional)

Toggle switch reveals a textarea to describe what changed in this revision. If filled, a revision history entry is created automatically.

After submission:

1. File is saved to uploads/ with a timestamped unique filename.
2. The original file is converted to PDF for in-browser preview (using ReportLab + python-docx/openpyxl/python-pptx).
3. A document record is inserted into the documents table with status Pending.
4. A confirmation email is sent to the uploader.
5. An approval request email with a secure review link is sent to the configured approver.
6. An in-app notification is created for the uploader and all admin users.
7. The upload is logged in the System Log.

6. Pending's (Approvals) — /approvals

Route: `approval_routes.py` → `index()`

Lists all documents in the system with their approval status. Accessible to all logged-in users.

Features:

Feature	Detail
Status filter tabs	All · Pending · Approved · Rejected
Search	Filter by file name or uploader
Review link	Each row has a "Review" button linking to the approval review page
Bulk approve/reject	Admin only — select multiple pending documents and approve or reject in one action
CSV Export	Downloads the current filtered list

7. Approval Review

Route: `approval_routes.py` → `review_document(token)`

Opened via a secure token link (from email or the Pending's page). Shows the full document metadata and an inline PDF preview.

- **Admins** see Approve and Reject buttons. Rejection requires a comment.
- **Non-admins** can view the document and its metadata but cannot take action.
- After a decision, the uploader receives an email notification and an in-app notification.
- The decision is logged in the System Log.
- The review token is signed with `itsdangerous` — it cannot be forged or tampered with.

8. Document Library — /document-library

Route: `document_library_routes.py` → `index()`

A structured, category-based file browser for reference documents. Unlike the Dashboard (which shows uploaded submissions), the Document Library organises files into a

predefined category tree defined in `data/document_library_data.py`.

Categories include: Procedures, CQ Manuals, Plant Procedures, Awards & Certifications, and more.

Navigation works by selecting a category from the left panel, then drilling into primary and secondary subcategories. Files can be viewed inline. All views are logged in the System Log.

9. Master Records (Plant Assets) — `/plant-assets`

Route: `plant_assets_routes.py` → `index()`

Browse documents organised by **Plant** → **Department**. The page loads plant cards; clicking a plant card fetches its departments via AJAX; clicking a department tab fetches the file list for that plant/department combination.

- All data is loaded dynamically via AJAX (`/plant-assets/departments` and `/plant-assets/files`).
 - File views are logged in the System Log.
 - The file list is read-only — no upload or delete from this page.
-

10. Customer Records — `/customer-records`

Route: `customer_records_routes.py` → `index()`

Browse documents organised by **Customer**. Works the same way as Master Records but grouped by customer (Hyundai, Tata, Ashok Leyland, TVS Motors, etc.) instead of plant.

- Customer list is loaded on page render.
 - Clicking a customer card fetches its files via AJAX (`/customer-records/files`).
 - File views are logged in the System Log.
-

11. Graphics Report — `/graphics-report`

Route: `graphics_report_routes.py` → `index()`

Visual analytics dashboard with Chart.js bar and doughnut charts.

Charts shown:

Chart	Description
Overall Status	Doughnut — Approved / Pending / Rejected counts
By Plant	Bar chart — document counts per plant, split by status
By Department	Bar chart — document counts per department
By Customer	Bar chart — document counts per customer

Summary stat cards below the charts show total documents, this month's uploads, and this week's uploads. All data is pulled live from the `documents` table.

12. Revision History — `/revision-history`

Route: `revision_history_routes.py` → `index()`

A chronological log of all document revisions. Each entry records:

Field	Detail
File Name	The revised document
Revision Number	e.g. Rev.01, Rev.02
Revised By	User name and ID
Plant / Department	Where the document belongs
Revision Date	Timestamp
Change Summary	Description of what changed

Filterable by Plant and Department. Revision entries are created automatically when a document is uploaded with the "Add revision summary" toggle enabled, or when an admin uploads a new version of an existing document.

13. Archive — `/archive`

Route: `archive_routes.py` → `index()`

Access: Admin, Manager, Supervisor, Approver only.

When a document is deleted from the Dashboard, it is not permanently removed — it is moved to the Archive table with its metadata preserved. The Archive page lists all soft-deleted documents.

- **Permanent Delete** button removes the record from the archive entirely and logs the action.
 - Regular users cannot access this page — they are redirected to the Dashboard.
-

14. System Log — `/system-log`

Route: `system_log_routes.py` → `index()`

Access: Admin, Manager, Supervisor, Approver only.

A full audit trail of every significant action in the system.

Logged actions:

Action	Trigger
LOGIN	User logs in
LOGOUT	User logs out
REGISTER	New account created
UPLOAD	File uploaded
VIEW	Document opened
DELETE	Document deleted
APPROVED	Document approved
REJECTED	Document rejected
APPROVAL_EMAIL	Approval request email sent
PASSWORD_CHANGE	User changes their password

Each log entry records: timestamp, user name, user ID, action type, and a details string. The log is filterable by action type using tab buttons at the top of the page.

15. People — /people

Route: `people_routes.py` → `index()`

Access: Admin, Manager, Supervisor, Approver only.

A directory of all registered users in the system. Displays each user's:

- Profile photo (or initial avatar if no photo uploaded)
- Full name
- Employee ID
- Email address
- Plant
- Department
- Role (with colour-coded badge)
- Account creation date

A live search box filters the table client-side by any field (name, email, role, department, etc.).

16. Profile — /profile

Route: `profile_routes.py` → `index()`

Each logged-in user's personal page. Shows their account details (read-only) and two action panels:

Profile Photo

- Upload a new photo (JPG, PNG, GIF, WebP).
- Photo is saved to `static/avatars/` named by user ID.
- Appears in the topbar and on the People page.

Change Password

- Hidden by default, revealed by a toggle button.
- Requires current password verification before accepting a new one.
- New password must be at least 8 characters.
- Password change is logged in the System Log.

Activity Log

- Right panel shows all System Log entries for the current user — their personal audit trail.

17. About DMS — /about

Route: `about_routes.py` → `index()`

An informational page explaining what Smart DMS is, how to use it, and a guided tour of all features. Includes a step-by-step walkthrough of the upload → approval → archive workflow.

18. About Company — /about/company

Route: `about_routes.py` → `company()`

A detailed company profile page for **ZF Rane Automotive India Private Limited**, including:

- Company overview and CIN
- Division details (SGD — Steering Gear Division, OSD — Occupant Safety Division)
- Plant locations and products
- Customer list
- Certifications (IATF 16949, ISO 14001, OHSAS 18001, FORD Q1)
- Awards (Deming Prize, Japan Quality Medal, CII Kaizen Gold)
- Engineering tools (CATIA, SolidWorks, COSMOS, AMESim)

Role & Access Control

Role	Dashboard	Upload	Approve/Reject	Archive	System Log	People	Delete Docs
User	✓	✓	View only	✗	✗	✗	✗
Approver	✓	✓	✓	✓	✓	✓	✓
Supervisor	✓	✓	✓	✓	✓	✓	✓
Manager	✓	✓	✓	✓	✓	✓	✓

Role	Dashboard	Upload	Approve/Reject	Archive	System Log	People	Delete Docs
Admin	✓	✓	✓	✓	✓	✓	✓

All users can view documents from all plants and departments. Department-based filtering is available as a UI convenience but does not restrict access.

Database Tables

All tables live in `smart_dms.db`. User credentials are stored separately in `data/smart_dms_users.sqlite3`.

Table	Purpose
<code>documents</code>	All uploaded document records with metadata and approval status
<code>document_versions</code>	Version history for each document (every upload creates a version entry)
<code>archive</code>	Soft-deleted documents moved here when deleted from the dashboard
<code>category_documents</code>	Documents uploaded directly to the Document Library
<code>revision_history</code>	Revision change log entries
<code>system_logs</code>	Full audit trail of all user actions
<code>notifications</code>	In-app notifications per user
<code>recently_viewed</code>	Last 10 documents viewed per user
<code>bookmarks</code>	Bookmarked documents per user
<code>users</code> (users.sqlite3)	User accounts, roles, plant, department, password hashes

Services Reference

Service	Responsibility
<code>AuthService</code>	Login, logout, register, session management, role checks

Service	Responsibility
UserStoreService	CRUD on the users SQLite database, avatar updates, password updates
DocumentService	Save uploads, fetch/filter documents, delete, approval status updates, review tokens
DocumentLibraryService	Resolve category keys, serve category tree data
CategoryDocumentService	Save and fetch documents stored in the Document Library
PlantAssetService	Fetch plant list, departments per plant, files per plant/department
CustomerRecordService	Fetch customer list and files per customer
RevisionHistoryService	Add and fetch revision history entries
DocumentTrackingService	Track recently viewed, bookmarks, upload trend data, stats summary
DocumentPreviewService	Build preview metadata (iframe/image/text mode) for a given file
PdfConversionService	Convert DOCX/XLSX/PPTX to PDF using ReportLab for in-browser viewing
SystemLogService	Write audit log entries for all action types
NotificationService	Create, fetch, mark-read in-app notifications
MailService	Send password reset, upload confirmation, approval request, and decision emails
PasswordResetService	Generate and verify time-limited signed password reset tokens

Environment Variables

All variables are optional — defaults work for local development.

Variable	Default	Purpose
SECRET_KEY	smart-dms-secret-key-change-in-production	Flask session signing

Variable	Default	Purpose
REVIEW_TOKEN_SALT	smart-dms-approval-review	Approval token signing
MAIL_SERVER	smtp.gmail.com	SMTP host
MAIL_PORT	587	SMTP port
MAIL_USE_TLS	true	Enable TLS
MAIL_USERNAME	(empty)	SMTP username
MAIL_PASSWORD	(empty)	SMTP password
MAIL_DEFAULT_SENDER	(empty)	From address
APPROVAL_RECIPIENT	(empty)	Email address that receives approval requests

If mail is not configured, password reset links are printed to the terminal.

Dependencies

Package	Version	Used For
Flask	3.1.3	Web framework
Flask-Mail	0.10.0	Email delivery
itsdangerous	2.2.0	Signed tokens (sessions, review links, password reset)
Werkzeug	3.1.8	Password hashing, file utilities
python-docx	1.2.0	Read DOCX files for PDF conversion
openpyxl	3.1.5	Read XLSX files for PDF conversion
python-pptx	1.0.2	Read PPTX files for PDF conversion
reportlab	4.5.1	Generate PDF files from Office documents
Pillow	12.2.0	Image handling for avatar uploads

Install all with:

```
pip install -r requirements.txt
```